Convolutional Autoencoders for One Class Anomaly Detection

By Tara Golonka and Alanna Cunha

***Abstract*—** Anomaly detection in images is a critical domain in a range of fields from manufacturing quality control to medical diagnostics, where detecting small deviations in data is required. This works aims to address the one class classification problem on three unique datasets: rotini pasta, roofing screws and capsules. We approach this problem using convolutional autoencoders (CAEs). In developing the final solution various convolutional layers were tested with both Mean Squared Error (MSE) and Structural Similarity Index Measure (SSIM) as the reconstruction loss functions. The most consistent algorithm was selected and additional techniques were added including denoising and augmenting the autoencoder to be variational (VAE) and beta (β-VAE). After extensive testing the best performance across all the datasets were found using a β-VAE with a β value of 0.25 and a latent vector dimension of 16.

Keywords—Convolutional Autoencoder, Beta Variational Autoencoder, Denoising, Latent Space, AUROC score

I. Introduction

Anomaly detection is the ability to detect outliers in data. This is an important domain with applications across many distinct fields including quality control in manufacturing, medical diagnostics, and cybersecurity. This application focuses on anomaly detection in images. Three datasets are used: rotini pasta, roofing screws and capsules. Each has a set of nominal training images and a set of test data with nominal and anomalous images. Thus, anomaly detection is done using one class classification. See **Fig. 1.** to **6.** of examples from each dataset.

All the datasets are unique and contain different types of anomalies. The pasta anomalies include incorrect pasta shape, string and pills see **Fig. 2**. Whereas for roofing screws anomalies range from subtle outliers such as upside-down washers to batteries instead of screws to tape on the screw. Capsules are all the same orientation and size, and anomalies include structural damage or misprints of '500'. A single algorithm needed to be applied to these distinct datasets which made the problem original and challenging to address.

Fig. 1. Nominal Images of Rotini Pasta

Fig. 2. Anomalous Images of Rotini Pasta

Fig. 3. Nominal Images of Roofing Screws

Fig. 4. Anomalous Images of Roofing Screws

Fig. 5. Nominal Images of Capsules

Fig. 6. Anomalous Images of Capsules

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

II. Literature Review

Many convolutional autoencoder techniques were explored and tested in the early stages to develop the final solution. Initially we looked at various designs for the convolutional layers in the network. A range of convolutional layers and hyperparameters were tested with foundational structures from other works providing a helpful base. Some of the configurations tested included changing the amount of downscaling in the network, adding batch and instance normalization and changing the activation functions from rectified linear units (ReLUs) to leaky rectified linear units (LeakyReLU). One approach that was replicated from another work included a nine-layer convolutional network with LeakyReLUs with slope 0.2 applied as activation functions after each layer, except for the output layers of both the encoder and the decoder which linear activation functions were used [1]. Modifications were made as necessary to fit our datasets. For example, this work was applied to images with 128x128 pixel inputs, whereas ours was a 224x244 input, thus implementing this required a larger final convolution.

This method was refined through iterative testing and suggestions from related work. The most consistent convolutional architecture was selected for further development and extended with additional techniques, including denoising and variational approaches.

The use of variational autoencoders was initially motivated by [2], which introduced the concept of getting the model to learn a distribution rather than fixed encodings, enforcing more smoothness and structure in the latent space. This enables smoother interpolation and improved generalization. The more modern formulation, β-VAE [3], was chosen to be implemented, as it improves on the standard VAE by introducing a weighting factor on the KL-divergence term to encourage more interpretable latent representations.

Foundational concepts from [4] motivated the exploration of denoising autoencoders. This approach was selected from the options detailed in the text due to its straightforward implementation and its ability to encourage the model to learn more robust structural features by reconstructing clean inputs from corrupted inputs. Among the methods explored, the β-VAE autoencoder ultimately produced the best results.

III. Proposed Approach

For our proposed approached we implemented a convolutional autoencoder architecture with LeakyReLU activation functions and instance normalization between layers, and augmented with a β -Variational Autoencoder (β -VAE).

A convolutional autoencoder is a neural network that learns to compress and reconstruct images by extracting spatial features using convolutions. It consists of two parts: an encoder that maps an input image into a lower dimensional latent space, and a decoder that reconstructs the image. In the encoder, each layer applies a convolution followed by an activation function. Down sampling is done using strides or pooling to progressively reduce spatial resolution while increasing feature depth. The decoder mirrors this process, using transposed convolutions to reconstruct the original spatial dimensions. The network is trained by minimizing reconstruction loss between the input and output through backpropagation.

In creating the autoencoder network several key decisions needed to be made regarding the depth of the network, type of pooling layers, convolutional layer parameters, output layer parameters and hyperparameters. The final solution was a 4-layer convolutional network. Each layer had a stride of 2 with LeakyReLU activation functions with slope 0.2 and instance normalization between them. Instance normalization was used since it normalizes each image independently, so it does not depend on batch size or dataset scale. This was found to yield better results than batch normalization, as it is more stable for small datasets and small batch training. With small datasets batch statistics can become noisy and unreliable [5].

This network was chosen through testing various configurations and through consulting other works. The decoder reversed this encoder process to reconstruct the image. Each layer also used a 3x3 kernel since this kernel size has a good balance between computational efficiency, parameter reduction, and the ability to capture complex local features. For example, using two 3x3 allows for more nonlinearity to be added with activation functions between layers than a single 5x5 kernel. **Fig. 7.** depicts this encoder network.

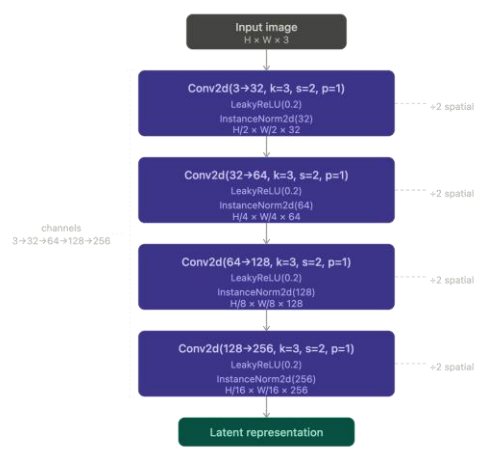

Fig. 7. Encoder Network

This network uses SSIM as the loss function. SSIM loss measures the perceptual similarity between two images by comparing their structure, luminance, and contrast instead of just pixel-wise differences. It is defined by the following equation:

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

In this equation, x is the original image, and y is the reconstructed image. The terms μ_x and μ_y represent the mean (average intensity) of each image, capturing luminance. The variances σ^2_x and σ^2_y measure contrast (how spread-out pixel values are), and σ_{xy} is the covariance between the two images, reflecting how well their structures align. The constants C_1 and C_2 are small stabilizing values added to avoid division by zero. SSIM outputs a value typically between 0 and 1, where 1 indicates perfect similarity. Since a higher value for loss corresponds to a worse reconstruction, 1−SSIM is used as the error.

Building upon this architecture, we add a probabilistic model: the Variational AutoEncoder (VAE). A standard convolutional autoencoder maps inputs to fixed, discrete vectors in the latent space, which often results in a discontinuous latent space that is poorly suited for generative sampling or smooth interpolation.

Instead of outputting a single latent vector, the final convolutional layer is flattened and passed through two parallel fully connected layers to predict the parameters of a multivariate Gaussian distribution: a mean vector μ and a log-variance vector log (σ^2). We use the log-variance over the standard deviation to ensure that our output maps to only positive variance values.

From these output parameters, a latent vector is sampled from the Gaussian distribution they define. To ensure differentiability during training, the reparameterization trick is applied. Instead of sampling directly from the learned distribution, a sample is drawn from a standard normal distribution and then scaled and shifted using the learned mean and variance parameters. This yields the following differentiable expression for the latent vector:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim N(0, I)$$

The variational autoencoder (VAE) further incorporates the Kullback-Leibler (KL) divergence into the loss function:

$$L_KL = D_KL(q(z|x) \,||\, p(z))$$

This term measures the difference between the learned latent distribution and a prior distribution, typically a standard normal distribution. It regularizes the latent space, encouraging smoothness and continuity.

The β-VAE extends this formulation by introducing a weighting parameter β that scales the KL divergence term. This controls the strength of the regularization and allows for adjustment between reconstruction accuracy and latent space structure.

$$L = L_recon + \beta \cdot D_KL(q(z|x) \,||\, p(z))$$

However, a large KL divergence weight can overly smooth the latent space, causing the model to lose important data variations and reducing reconstruction fidelity.

IV. Results and Evaluation

To determine the hyperparameters of our convolutional network including number of convolutional layers, number of filters, bottleneck size, activation functions, loss functions and learning rates extensive testing was done.

AUROC scores from preliminary testing of three different convolutional layer configurations are described in **Table 1**. Design 1 was a 4-layer convolutional network. Each layer had a stride of 2 with ReLU activation function between layers. Design 2 was the network described in the proposed solution, before the SSIM loss function and the additional features were added. This design incorporated parts of both of design 1 and 3. Design 3 was the 9-layer network described in the literature review closely following the paper [1].

Table 1. AUROC Scores for Initial Network Designs

	Screws	Pasta	Capsules
Design 1	0.48	0.55	0.44
Design 2	0.52	0.62	0.62
Design 3	0.52	0.43	0.64

From the testing of these various layer configurations in the network, design 2 was selected to be developed further because of the more consistent, promising scores. Our next step was to implement SSIM as the loss function and compare to MSE. **Table 2** describes results using each of these loss functions. Averaged between all the datasets the performance was increased by using SSIM over MSE. Individually though SSIM improved the performance for screws and pasta but was worse for capsules. SSIM likely performs better on pasta and screws because these objects have complex textures and varying shapes, so capturing structural similarity is more important than exact pixel matching. In contrast, capsules are smooth and consistently oriented, so small pixel-level differences are meaningful, making MSE more effective. SSIM was ultimately chosen to continue developing since it was more effective for a larger portion of the overall datasets.

Table 2. AUROC Scores for MSE vs SSIM Loss Functions

	Screws	Pasta	Capsules
MSE	0.52	0.62	0.62
SSIM	0.64	0.70	0.53

Building upon the foundational architecture of design 2 and the chosen SSIM function, further testing was conducted to find the best setup for each autoencoder and to compare both designs. We analyzed a Baseline Autoencoder, a Denoising Autoencoder (DAE), and a β -Variational Autoencoder (β -VAE). Hyperparameter searches focused primarily on the latent

space dimensionality and the noise factor for the DAE and the β value for β -VAE.

The benchmark hyperparameters for this testing were epochs = 30 for all autoencoders, learning rate = 1e-3 for the DAE, and learning rate = 1e-4 for the β -VAE. A learning rate any higher for the β -VAE would yield gradient explosion. A basic autoencoder with epochs = 30, learning rate = 1e-3, and latent dimensions of 16 yielded a ceiling AUROC score of 0.56.

A hyperparameter search for noise factor of the DAE over (0.2, 0.4, 0.8) in **Table 3** revealed that a factor of 0.4 achieved the optimal balance of regularization, peaking at an average AUROC of 0.55. At this noise level, restricting the latent dimension to 8 marginally outperformed a dimension of 16 (0.55 vs. 0.54).

Table 3. AUROC Score for Noise Factor

Noise Factor	Screws	Pasta	Capsules	Average
0.2	0.65	0.18	0.67	0.50
0.4	0.67	0.32	0.67	0.55
0.8	0.64	0.23	0.67	0.51

The β -VAE introduced a Kullback-Leibler (KL) divergence penalty to adjust how much to penalize reconstruction accuracy over latent space regularization. A search over β (0, 0.25, 0.5, 1.0) shown in **Table 4**, identified β = 0.25 as the optimal value over all three data sets. Increasing the latent dimension from 8 to 16 at β = 0.25 improved the overall average AUROC from 0.6 to 0.64, marking it the highest quantitative performance achieved in this study.

Table 4. AUROC Score for Noise Factor

β	Screws	Pasta	Capsules	Average
0	0.51	0.71	0.50	0.57
0.25	0.63	0.57	0.59	0.6
0.5	0.63	0.54	0.48	0.55
1	0.63	0.54	0.48	0.55

The general success of the DAE and the β -VAE led to the testing of a joint design, β -DVAE. Adding input noise into the optimized β -VAE resulted in a performance degradation, dropping the average AUROC from 0.6378 to 0.5800. This shows that the representation learnt by the β -VAE is not helped by the addition of noise. This led to the final design of the β -VAE (Latent 16, β -VAE=0.25).

Looking at some more qualitative results of the DAE and the β -VAE, the Denoising Autoencoder generally outperforms the β -VAE in preserving structural integrity and color across these distinct datasets.

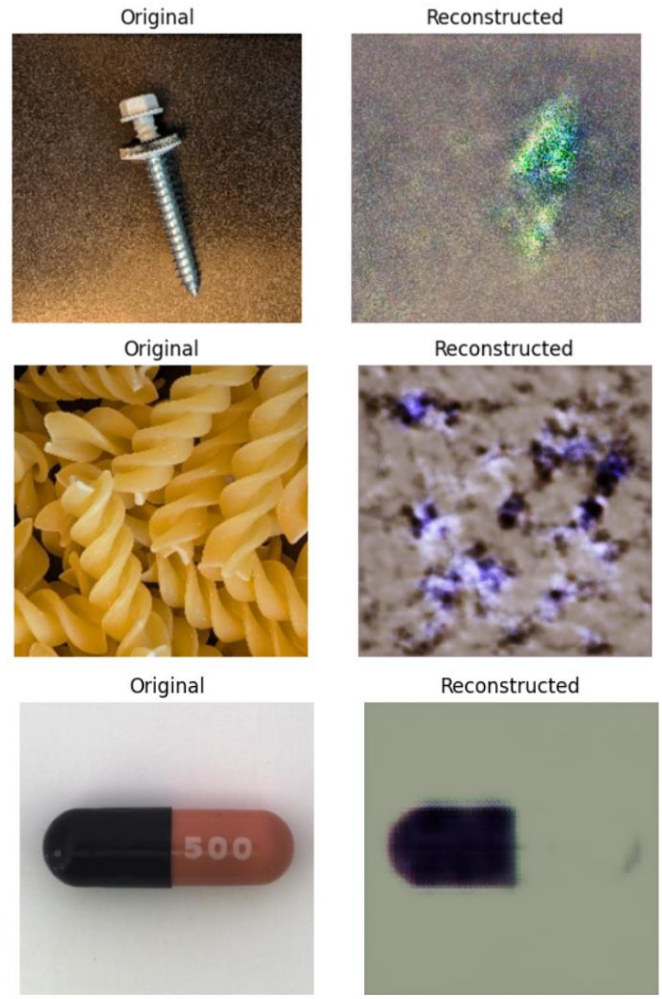

Fig. 8. Input and reconstructed images from DAE

While the DAE manages to capture vague spatial representations and colors for both the screw and pasta images, the β -VAE outputs only random noise, see **Fig. 8.** and **Fig. 9.**

Fig. 9. Input and reconstructed images from β -VAE

Interestingly, both models perform well on the capsule image, successfully retaining its basic shape and distinct division in the middle. However, the β -VAE far outperforms, in spite of weird noise artifacts.

V. Conclusion

This work explored different convolutional autoencoder networks for image anomaly detection. In developing the final solution various convolutional layers were tested with both MSE and SSIM as the reconstruction loss functions. The most consistent algorithm was selected, and additional techniques were added including denoising and augmenting the autoencoder to be variational and β-VAE. After extensive testing the best performance across all the datasets were found using a β-VAE with a β value of 0.25, a latent vector dimension of 16 and SSIM as the loss function. Our results and evaluation show the process of how we got these values and this final solution. Notably, the qualitative results seem to indicate a better performance by the DAE, despite the higher AUROC scores of the β-VAE**.** Overall, implementing a convolutional autoencoder and testing many different designs and hyperparameters yielded interesting results which taught us about anomaly detection and model training and testing.

References

[1] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, "Improving unsupervised defect segmentation by applying structural similarity to autoencoders," arXiv preprint, 2018, doi: 10.5220/0007364503720380.

[2] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *arXiv preprint arXiv:1312.6114*, 2013

[3] I. Higgins, L. Matthey, A. Pal, C. P. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[5] V. Thakkar, S. Tewary, and C. Chakraborty, "Batch normalization in convolutional neural networks—A comparative study with CIFAR-10 data," in Proc. 5th Int. Conf. Emerging Applications of Information Technology (EAIT), Kolkata, India, 2018, pp. 1–5, doi: 10.1109/EAIT.2018.8470438.